

Definição

Requisitos adicionais para o projeto

- Precisa conter porções teóricas e práticas
- A porção teórica deve ser baseada em artigos científicos (tipicamente 2-3)

Referências

- [Load Balancing Evaluation Tools for a Private Cloud: A Comparative Study \(2018\)](#)
 - compara diferentes ferramentas para avaliação de algoritmos de balanceamento de carga em ambientes de nuvem privada
- [Comparative Analysis of Selected Open-Source Solutions for Traffic Balancing in Server Infrastructures Providing WWW Service \(2021\)](#)
 - compara soluções de código aberto para balanceamento de tráfego em servidores web
- [Round-robin Algorithm in HAProxy and Nginx Load Balancing Performance Evaluation: a Review \(2018\)](#)
 - revisão da literatura sobre o algoritmo Round Robin implementado no HAProxy e no Nginx
- [Performance And Fault Tolerance Analysis Of Load Balancing Strategies In Microservices Architecture With Controlled Failure Injection \(2026\)](#)
 - analisa o desempenho dos algoritmos Round Robin e Least Connection no HAProxy
- [Optimizing Load Balancing Framework for a Distributed Local Network \(2026\)](#)
 - avalia o desempenho de diferentes algoritmos de balanceamento de carga em clusters de servidores web

Roteiro

Melhorando as perguntas:

- Pergunta 1 (Falhas - Ajustada): Quanto tempo o HAProxy/NGINX leva para detectar e remover do cluster uma réplica que sofreu um crash abrupto (via Passive vs Active Health Checks)? Quantas requisições do usuário falham (HTTP 5xx) nesse intervalo?
- Pergunta 2 (Escalabilidade): Qual é o impacto no tempo de resposta (latency tail / percentis 95 e 99) e na vazão (throughput) quando o tráfego sofre um pico repentino (efeito Flash Crowd)?
- Pergunta 3 (Algoritmos - Nova): Como a escolha do algoritmo de balanceamento (ex: Round-Robin vs Least Connections) afeta a recuperação do sistema e a distribuição da sobrecarga residual após a queda de um nó?

Em prática

Fase 1: Arquitetura do Ambiente (A Infraestrutura)

- **Ferramenta:** docker-compose
- **Componentes:**
 - **1 Nó Balanceador:** HAProxy ou NGINX.
 - **3 Nós de Backend:** Réplicas de app web (FastAPI ou Node.js que simulem algum processamento).
 - **1 Gerador de Carga:** Um container separado rodando uma ferramenta de testes de estresse (como o **k6**, **Apache Benchmark** [Arthur usou k6 em segsis, mas esse apache parece interessante também]).

Fase 2: Mecanismos de Observabilidade (Medição)

- Configurar o gerador de carga para salvar os resultados em tempo real (tempo de resposta por requisição, códigos de status HTTP).
- Monitorem os logs do HAProxy/NGINX para capturar o timestamp exato em que ele marca um nó como DOWN. -> Não

sabemos exatamente com o que vamos fazer isso (usar alguma ferramenta que gere gráficos seria uma boa)

Fase 3: Roteiro dos Experimentos

Para obter conclusões científicas (exigidas no projeto), isolar as variáveis:

- **Cenário A (Linha de Base):** Rodar o gerador de carga com tráfego constante e linear. Medir a vazão máxima estável do sistema sem falhas.
- **Cenário B (Injeção de Falhas):** Durante um fluxo estável de requisições, matar abruptamente um dos containers de backend (ex: docker kill backend-02) ou usar alguma ferramenta para causar outro tipo de instabilidade ([pumba](#)).
 - *O que analisar:* Quantos segundos o balanceador demora para parar de enviar tráfego para lá? Quantos erros foram retornados ao cliente?
- **Cenário C (Pico de Tráfego):** Simular uma rampa agressiva de usuários simultâneos (ex: saltar de 10 para 1000 requisições por segundo em 5 segundos).
 - *O que analisar:* O balanceador começa a enfileirar requisições? Há aumento drástico no tempo de resposta?

Fase 4: Análise Teórica e Relatório

Cruzar esses dados práticos com as referências escolhidas. Por exemplo, verificar se o comportamento que vocês observaram no HAProxy bate com o que o artigo de análise de tolerância a falhas de 2026 descreve.

Divisão de trabalho

Arthur: Realizar a análise para o cenário A

Rafael: Realizar a análise para o cenário B

Isaque: Realizar a análise para o cenário C

Heitor: Parte teórica e plot dos gráficos (Pandas + seaborn/matplotlib parecem ser as melhores opções)

Repositório

[sd-load-balancers](#)